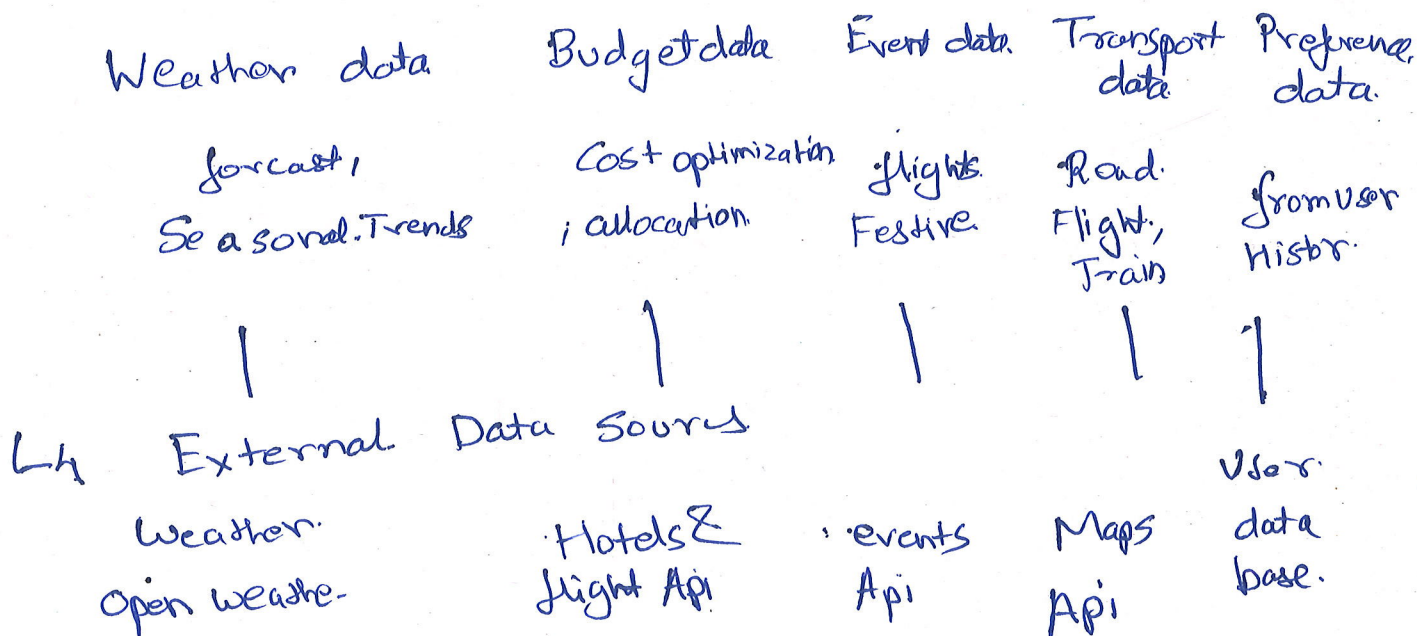
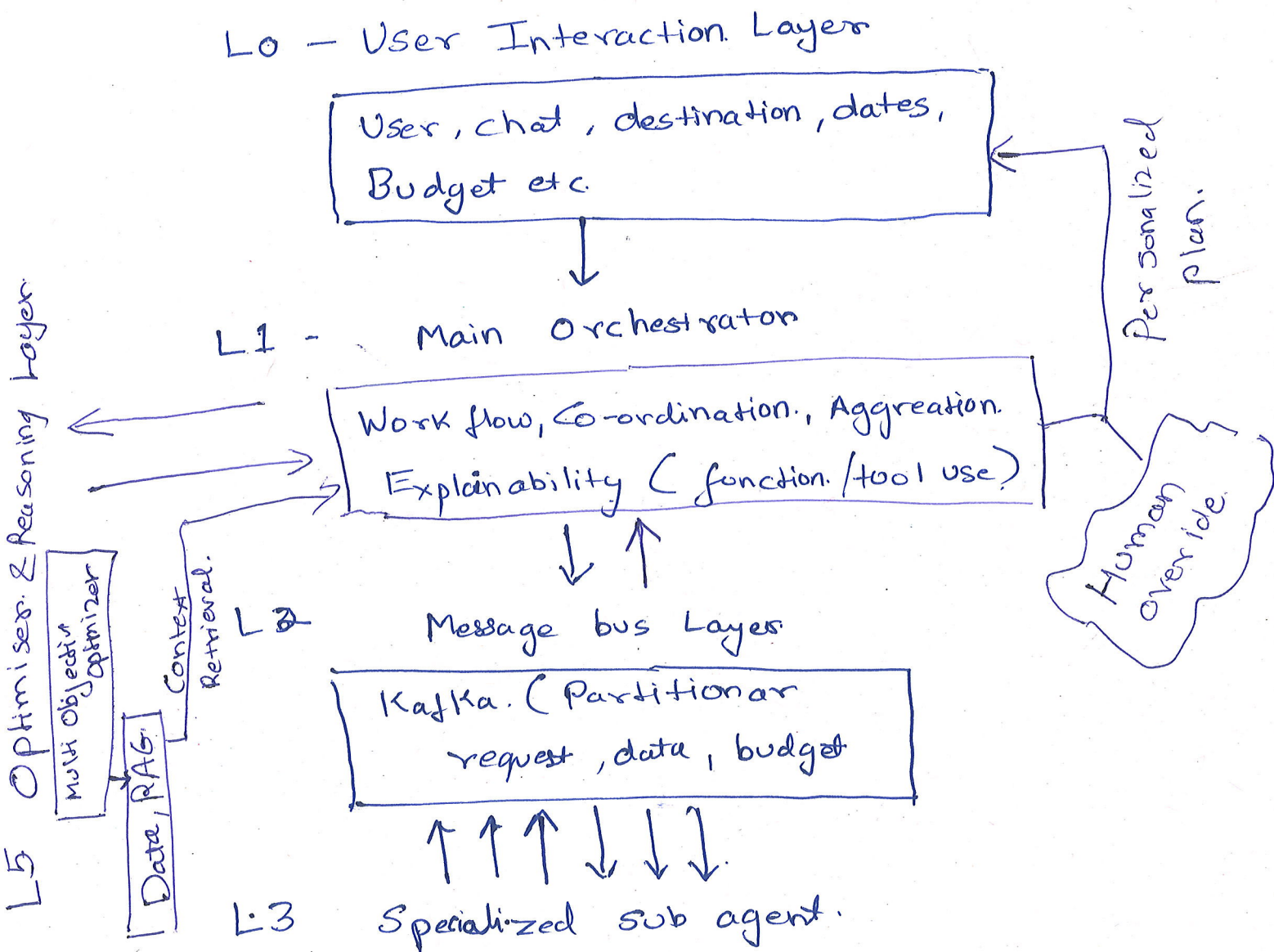


Part 3 System Architecture



2. Use Case flow

Simple Case - Weather-Based adjustment
Scenario - User request a Goa trip in July.

Flow:

1. User → MainAgent - "plan 5 day Kerala trip"
2. MainAgent → Weather agent
↓
weather api = heavy rain forecast
3. MainAgent → optimizer → suggest other Season or destination
4. optimizer → Main agent → user

3. Advance Use Case - Dynamic Re-planning (User-triggered.)

Scenario - traveler informs system that flight is delayed

Flow

1. User → MainAgent Initial plan generation
2. User → MainAgent "My flight is delayed by X hours"
3. MainAgent (LLM Reasoning) (Main Agent classifier as a disruption agent event & Extract delay duration)
4. MainAgent → MessageBus publishes transport event {delay: Xhours, flight}
5. MessageBus → Transport Agent
Budget Agent
Event Agent Relevant sub-agent & start recalculating their part
6. Re-processing by Allagent reconfirms, reestimated reorders activities
7. Sub agent → Message bus
→ Main Agent Each agent send Message to message bus, Main agent
8. Re-optimization
Main Agent → optimizer agent optimizer triggered. revised Context Cost, Comfort
9. Validation Main Agent
↔ Allagent All agents validates Relevant API

Memory Management

① Short term

We save it in cache/buffer

— Tools Redis

② Episodic memory

Saved in postgres

(It is summarized with LLM & then saved)

③ Long term memory

④ Global Memory

TTL

(min-hours)

Decide in days

Vector dB/Graph (No TTL)

Events

User starts chat

Trip planning completed

Weekly job

Saved memory

can be used across agent

New Session

Memory

STM (Redis)

STM → EM (postgres/sql)

EM → LTM embedding (Vector dB)

LTM summary → GM Fact.

STM uses (LTM + GM context)

3. PEAS Framework

A	Performance	Trip Satisfaction, budget allocation. real-time. adaption.
B	Environment	Api,s — weather, flight, Events, User data.
C	Actuators	LLM Reasoner, Notification System
D	Sensors	Weather feeds, flight Api, Feed back

4. Agent Collaboration Model

Communication Mechanism

→ Publish — Subscribe. via Kafka.

→ Json events with fields:

```
{ "Customer ID": "path Honors class.",  
  "Topic": "Weather.data",  
  "from": " ",  
  "to": " ",  
  "Payload": {  
    " " : " ",  
    "Confidence": 0.9,  
    " " : " "  
  }  
}
```

Main agent → Kafka.

Kafka → Subagents Consume process

Subagents → Kafka. → Main agent

Main agent aggregates context & invokes Optimizer

Optimizer computes plan & return to Main Agent.

5. AI Layer

Role Layer	Constraints budget, time
LLM Layer	Reasoning & Summarization
Optimization	multi-objective trade off Cost vs Comfort

6. Ethical & Responsible Design

Privacy — Encryption

Explainability — Provide Reason in reply

Transparency — Confidence Scores

Accountability — Human override & Logging

7. Future enhancements

Predictive forecasting

(Use External service.)

ML Model will overkill system)

GenAI Summarization

LLM can suggest daily plan & what will be needed.

Part 4 Evaluation Criteria

- ① System thinking — Hierarchical multi agent Arch.
- ② Autonomy — Agents operate independently via Kafka
- ③ Technical depth — Use of API, Message bus, ML
- ④ Reasoning — Context aware LLM & multi-objective
- ⑤ Ethics & UX — Privacy by design & explainable output)